

Módulo 10 – Capítulo 05 – Sección 01

Model drift: data drift vs concept drift y cómo detectarlos estadísticamente

Un modelo que funcionaba bien cuando fue desplegado puede degradarse gradualmente en producción sin producir ningún error de sistema: responde a todos los requests con HTTP 200, la latencia es estable, la GPU utilization es normal. El único síntoma es que las predicciones son cada vez peores —y ese síntoma es invisible para los sistemas de monitoreo de infraestructura convencionales. El drift de modelos es este fenómeno: el deterioro de la calidad predictiva de un modelo debido a cambios en el entorno de producción que el modelo no anticipó durante el entrenamiento. Entender la taxonomía del drift —qué tipos existen, qué los causa, y cómo detectar cada uno— es el prerequisite para construir un sistema de monitoreo que detecte estos fallos silenciosos antes de que sean evidentes para los usuarios.

El **data drift** (también llamado covariate shift) ocurre cuando la distribución de los inputs que llegan al modelo en producción —las features de entrada— cambia respecto a la distribución que el modelo vio durante el entrenamiento. La relación entre inputs y outputs permanece estable (el modelo sigue siendo correcto para los datos que conoce), pero una fracción creciente de las requests en producción cae en regiones del espacio de features que el modelo nunca vio en training, produciendo predicciones menos confiables. Un ejemplo concreto: un modelo de recomendación de e-commerce entrenado con datos de 2023 no anticipó los cambios de comportamiento de los usuarios durante el período de alta inflación de 2024; las features de comportamiento de compra tienen distribuciones muy diferentes a las del periodo de entrenamiento. El data drift es detectable sin ground truth (sin labels reales) porque solo requiere comparar la distribución de los inputs actuales con la distribución histórica de referencia.

El **concept drift** es más difícil de detectar y más peligroso porque requiere ground truth para identificarse. Ocurre cuando la relación subyacente entre inputs y outputs cambia, aunque la distribución de los inputs permanezca estable. Usando el mismo ejemplo: en el modelo de recomendación de e-commerce, el concept drift ocurriría si los mismos patrones de comportamiento de usuario que antes predecían conversión ahora predicen solo browse sin

compra, porque cambió la economía o la competencia. El modelo no vio ningún cambio en los inputs, pero los outputs correctos son ahora diferentes. El concept drift solo es detectable comparando las predicciones del modelo con los outcomes reales (labels) de producción, lo que implica que el sistema de captura de feedback —sea human-in-the-loop, labels implícitos por comportamiento posterior, o anotadores— debe estar diseñado desde el inicio del sistema.

El tercer tipo de drift, el **label drift** (prior probability shift), ocurre cuando cambia la distribución de los outputs esperados independientemente de los inputs. En un modelo de clasificación binaria de fraude, si la tasa de fraude real en producción sube del 1% al 5% porque hay una nueva campaña de fraude activa, el modelo calibrado sobre la distribución histórica producirá un número insuficiente de detecciones positivas aunque su arquitectura sea correcta. El label drift es detectable en modelos de clasificación monitorizando la distribución de las predicciones del modelo en producción: si el ratio de predicciones positivas se desvía significativamente del ratio histórico, puede indicar label drift aunque los inputs parezcan estables.

Conceptos clave de detección de drift

- **Data drift (covariate shift):** cambio en $P(X)$ con $P(Y|X)$ estable; detectable estadísticamente comparando distribuciones de features entre producción y datos de entrenamiento de referencia usando KS-test o Chi-squared; no requiere ground truth.
- **Concept drift:** cambio en $P(Y|X)$ independiente de cambios en $P(X)$; detectable solo con ground truth mediante ventanas deslizantes de métricas de calidad (accuracy rolling 7-day, 30-day); el sistema de feedback es imprescindible.
- **Label drift (prior probability shift):** cambio en $P(Y)$ sin cambio en $P(X|Y)$; detectable en modelos de clasificación monitorizando la distribución de las predicciones del modelo en producción; señal indirecta de concept drift.
- **KS-test implementation:** para cada feature continua, calcular la estadística $D = \max|F_1(x) - F_2(x)|$ y el p-value; alertar cuando $p\text{-value} < 0.05$ en N features simultáneas, ajustando por corrección de Bonferroni para controlar los falsos positivos en datasets con muchas features.
- **Ventanas de detección:** drift de corto plazo (últimas 24h vs referencia) para detectar cambios abruptos; drift de largo plazo (último mes vs referencia de 6 meses atrás) para detectar deriva gradual que las ventanas cortas no capturan.

Nota del Arquitecto: El data drift puede detectarse sin ground truth comparando distribuciones de inputs, pero el concept drift requiere inevitablemente labels reales.

Diseñar el sistema de captura de feedback desde el inicio —sea a través de conversiones medidas, clicks, ratings explícitos o anotadores humanos— es tan importante como el modelo en sí. Los sistemas de ML que no tienen un mecanismo de feedback son ciegos al concept drift hasta que el impacto en el negocio es visible, momento en el que el problema lleva semanas o meses acumulándose.

El $\text{PSI} > 0.2$ en una feature crítica o el $\text{p-value} < 0.05$ en múltiples features simultáneas son las señales que deben disparar la investigación de drift y potencialmente el reentrenamiento del modelo. La siguiente sección examina las métricas específicas para medir la calidad de LLMs en producción, que plantean desafíos distintos de los modelos de ML clásico porque no existe una única métrica escalar equivalente al accuracy.

Módulo 10 – Capítulo 05 – Sección 02

Métricas de calidad: coherencia, fidelidad y relevancia medidas en producción

Medir la calidad de un LLM en producción es estructuralmente más complejo que medir la calidad de un modelo de clasificación o regresión. En un modelo de clasificación, el accuracy es un número escalar calculable automáticamente siempre que se tengan los labels correctos: o la predicción coincide con el label o no coincide. En un LLM, la calidad de una respuesta es multidimensional, subjetiva y dependiente del contexto: una respuesta puede ser gramaticalmente correcta pero factualmente incorrecta, o puede ser factualmente correcta pero irrelevante para la pregunta del usuario, o puede ser relevante pero internamente contradictoria. Las métricas offline como BLEU o ROUGE, aunque útiles para comparaciones controladas en benchmarks, tienen una correlación modesta con la percepción de calidad de usuarios reales en sistemas de producción.

Las tres dimensiones de calidad más operacionalmente relevantes para LLMs en producción son **coherencia**, **fidelidad** y **relevancia**. La coherencia mide si la respuesta es internamente consistente y libre de contradicciones: un LLM que afirma en el primer párrafo que "el producto tiene garantía de dos años" y en el tercer párrafo que "la garantía expira en 12 meses" produce una respuesta incoherente independientemente de cuál versión sea la correcta. La fidelidad (o factual accuracy o grounding) mide si las afirmaciones del modelo son correctas respecto a los datos de referencia disponibles: en sistemas RAG (Retrieval Augmented Generation), el 100% de las afirmaciones del modelo deberían ser respaldables por los documentos recuperados, y cualquier afirmación que no tiene soporte documental es potencialmente una alucinación. La relevancia mide si la respuesta aborda la pregunta o tarea del usuario de forma apropiada: un modelo que responde correctamente a una pregunta diferente de la que se hizo no es útil, independientemente de la calidad técnica de su respuesta.

El enfoque más escalable para medir estas tres dimensiones en producción es el **LLM-as-a-judge**: usar un modelo más capaz (GPT-4, Claude) para evaluar las salidas del modelo de producción en una muestra estadísticamente representativa de las requests reales. El

evaluador recibe el prompt original, la respuesta del modelo de producción, y opcionalmente los documentos de contexto (para sistemas RAG), y produce una evaluación en escala 1-5 o binaria para cada dimensión de calidad. Muestrear el 1-5% de todas las llamadas en producción para evaluación automática con LLM-as-a-judge proporciona una señal de calidad continua con un costo operativo manejable: a 1M requests/día con 2% de muestreo son 20,000 evaluaciones diarias, cada una con un costo de algunos centavos dependiendo del modelo evaluador.

La evaluación humana directa es más precisa que el LLM-as-a-judge pero menos escalable: plataformas como Scale AI, Toloka o equipos internos de anotadores evalúan muestras de producción con protocolos de rúbrica definidos. La evaluación humana es indispensable para calibrar el sistema de LLM-as-a-judge (verificar que el juez LLM concuerda con evaluadores humanos en una muestra de validación), para categorías de calidad donde los modelos evaluadores tienen limitaciones conocidas (humor cultural, sensibilidad contextual específica del dominio), y para decisiones de governance donde la evaluación automatizada es insuficiente para las consecuencias regulatorias.

Métricas de calidad para LLMs en producción

- **Coherencia:** evaluada con LLM-as-a-judge en escala 1-5 ("¿es esta respuesta internamente consistente?"), o con NLI classifier (DeBERTa fine-tuned para Natural Language Inference) detectando auto-contradicciones entre sentencias de la respuesta.
- **Fidelidad/Grounding:** en sistemas RAG, porcentaje de afirmaciones del modelo respaldables por los documentos recuperados; medido con classifiers de factual consistency (minicheck, TRUE) que identifican claims no soportados por el contexto.
- **Relevancia:** similitud semántica entre la pregunta y la respuesta via embedding cosine similarity; o evaluación con LLM-as-a-judge del grado en que la respuesta aborda directamente la pregunta planteada.
- **Toxicidad y safety:** clasificadores especializados (Perspective API, Llama Guard, custom fine-tuned classifiers) aplicados a todas las salidas en producción para detectar contenido dañino; no se muestrea, se evalúa el 100% del tráfico.
- **Task completion rate:** para sistemas agentes o multi-turno, porcentaje de conversaciones que resultan en completión exitosa de la tarea del usuario; medido con análisis de conversaciones o feedback explícito del usuario (rating, marca de "resuelto").

Nota del Arquitecto: Muestrear el 1-5% de todas las llamadas en producción para evaluación automática con LLM-as-a-judge proporciona una señal de calidad continua y estadísticamente significativa con un costo operativo manejable. El error común es implementar la evaluación solo para casos problemáticos reportados por usuarios: eso detecta los fallos más obvios pero pierde los fallos sutiles y sistemáticos que afectan a la mayoría del tráfico silenciosamente.

Las métricas de calidad de LLMs en producción son más complejas de implementar que las métricas de ML clásico, pero son igualmente necesarias para detectar degradaciones antes de que afecten a los usuarios. La siguiente sección examina las métricas de performance de infraestructura —latencia, throughput y tasa de error— que son complementarias a las métricas de calidad del modelo y deben monitorizarse en paralelo.

Módulo 10 – Capítulo 05 – Sección 03

Performance monitoring: latencia, throughput y tasa de error en serving

El monitoreo de performance del serving de modelos combina las métricas de un sistema de software tradicional —latencia, throughput, error rate— con métricas específicas de IA que no tienen equivalente en servicios convencionales: GPU utilization, model loading time, KV cache utilization en LLMs. Monitorear solo las primeras sin las segundas es como monitorear una base de datos por número de queries sin medir el uso de índices: los síntomas de un problema pueden estar visibles en las métricas convencionales, pero las causas raíz requieren métricas específicas del sistema.

Las métricas de latencia para LLMs tienen una **estructura dual** que no existe en APIs convencionales. El **Time To First Token (TTFT)** mide el tiempo desde el envío del request hasta que el usuario recibe el primer token de la respuesta generada, y es el determinante principal de la percepción de responsividad: un usuario interactuando con un chatbot puede tolerar 30 segundos de generación total si el primer token aparece en 200ms, porque el stream continuo de texto indica que el sistema está funcionando. El **Time Per Output Token (TPOT)** mide la velocidad de generación continua después del primer token, y determina el tiempo total de respuesta para prompts con outputs largos. Los SLOs típicos en sistemas de producción son TTFT < 500ms para el percentil 95 y TPOT < 50ms/token para el percentil 95, aunque estos valores varían significativamente según el modelo (tamaño, arquitectura), el hardware (A100 vs L4 vs H100), y el tipo de aplicación (interactiva vs batch).

El throughput en sistemas de LLM tiene dos métricas complementarias. Los **requests per second (RPS)** miden la capacidad del sistema para aceptar nuevas peticiones y son el indicador más directo de la capacidad de atención del sistema. Los **tokens per second (TPS)** —agregados entre todos los requests en procesamiento simultáneo— miden la eficiencia en el uso del GPU y son el indicador correcto para evaluar si el batching está siendo efectivo. Un sistema que tiene un TTFT aceptable pero un TPS bajo está subutilizando el GPU porque no está ejecutando suficientes requests en paralelo; un sistema con TPS alto pero

TTFT elevado está sobreutilizando el GPU y poniendo requests en cola antes de empezar a procesarlos.

La tasa de error requiere una taxonomía específica para modelos en producción. Los **errores 4xx** (bad request, context length exceeded) indican problemas en las aplicaciones cliente que consumen el endpoint y no deben confundirse con problemas del modelo o la infraestructura. Los **errores 5xx** (timeout, OOM del servidor, error de Kubernetes) indican problemas de infraestructura que el equipo de plataforma debe investigar y resolver. Los **errores de calidad del modelo** —respuesta vacía, alucinación detectada por safety filter, respuesta que no respeta el formato esperado— son errores que el serving layer reporta como HTTP 200 desde el punto de vista de la infraestructura, pero que deben registrarse como errores de calidad en las métricas de modelo. Esta distinción es crítica: los equipos de operaciones y los equipos de ML deben monitorizar sus categorías de error respectivas con herramientas y procesos distintos.

Métricas de performance para serving de modelos

- **TTFT (Time To First Token):** latencia desde el envío del request hasta recibir el primer token de la respuesta; crítico para UX en aplicaciones interactivas; SLO típico p95 < 500ms; aumenta con el tamaño del contexto de entrada (prefill computation).
- **TPOT (Time Per Output Token):** milisegundos por token generado; determina la velocidad de generación; depende del tamaño del modelo, el hardware y el batch size del servidor de inferencia; un TPOT alto indica saturación de GPU.
- **GPU Memory Utilization:** porcentaje de VRAM ocupada por el modelo, el KV cache y los batches activos; mantener < 85% para evitar OOM en picos de tráfico no anticipados; picos frecuentes a 95%+ son señal de necesidad de más capacidad o mejor configuración de batching.
- **Error rate por tipo:** distinguir entre errores 4xx (bad request, context exceeded), 5xx (timeout, OOM del servidor), y errores de calidad del modelo (respuesta vacía, safety filter hit); causas raíz y remedios distintos para cada categoría.
- **Queue depth:** número de requests en cola esperando ser procesados; una queue creciente con latencia estable indica necesidad de más réplicas; una queue estable con latencia creciente indica degradación de throughput (posiblemente por fragmentación del KV cache o problemas de batch scheduling).

Nota del Arquitecto: El SLO de latencia de un LLM en producción debe definirse en términos de TTFT y TPOT separadamente, no solo en tiempo total de respuesta.

Un SLO de "tiempo total < 10 segundos" puede ser cumplido por un sistema que tarda 8 segundos en mostrar el primer token (TTFT terrible) y luego genera muy rápido (TPOT excelente), pero esa experiencia es inaceptable para usuarios interactivos. Los SLOs deben reflejar cómo los usuarios realmente experimentan el sistema.

Las métricas de performance de infraestructura y las métricas de calidad del modelo son dos capas complementarias del monitoreo de un sistema de IA: ambas son necesarias, ninguna es suficiente por sí sola. La siguiente sección examina cómo diseñar el sistema de alertas para estas métricas de forma que sea sensible a problemas reales sin producir alert fatigue que lleva a los ingenieros a ignorar las alertas.

Módulo 10 – Capítulo 05 – Sección 04

Alertas inteligentes: umbrales adaptativos vs estáticos para sistemas de IA

Una alerta que se ignora habitualmente es funcionalmente equivalente a ninguna alerta: si el equipo de on-call normaliza recibir 30 alertas por turno de las cuales 28 son falsos positivos, la alerta 29 que es real se perderá en el ruido. Este fenómeno —alert fatigue— es uno de los problemas operacionales más comunes en sistemas de IA en producción, y su causa raíz es frecuentemente la aplicación de umbrales estáticos diseñados para microservicios convencionales a métricas que tienen estacionalidad y varianza natural significativa. Los sistemas de IA inteligentes merecen sistemas de alertas inteligentes.

Los **umbrales estáticos** —alertar cuando la latencia p99 supera 2000ms o cuando el drift score KS supera 0.15— son insuficientes para métricas de sistemas de IA por razones distintas para cada tipo de métrica. Las métricas de infraestructura (latencia, throughput) tienen estacionalidad predecible: la latencia es sistemáticamente más alta los lunes por la mañana cuando los usuarios vuelven al trabajo, más alta los días de campaña de marketing, y más baja los domingos. Un umbral fijo calibrado para evitar alertas en horas pico generará falsos negativos durante las horas de menor tráfico cuando el sistema bajo el mismo umbral absoluto está operando de forma degradada. Las métricas de calidad del modelo tienen un problema diferente: el nivel "normal" de drift score o de LLM-judge score cambia después de un reentrenamiento o un cambio de modelo, y el umbral fijo calibrado para el modelo anterior puede ser incorrecto para el nuevo modelo desde su primer día de despliegue.

Los **umbrales adaptativos** calculan dinámicamente el umbral de alerta basándose en el comportamiento histórico reciente de la métrica. Grafana Alerting soporta expresiones de alerta basadas en percentiles de una ventana temporal: una regla como `alert if current_value > percentile(95, metric[7d])` alerta solo cuando el valor actual supera el percentil 95 del comportamiento de los últimos siete días, ajustándose automáticamente a los patrones estacionales sin intervención manual. Para métricas con estacionalidad semanal pronunciada, Meta's Prophet puede modelar el comportamiento esperado para cada hora de cada día de la semana y generar umbrales que se ajustan automáticamente al contexto

temporal. Tras un reentrenamiento del modelo, el umbral adaptativo se recalibra automáticamente sobre el comportamiento del nuevo modelo en las primeras horas de producción, sin requerir actualización manual por el equipo de operaciones.

Para detectar degradaciones lentas y sostenidas que los tests estadísticos estándar (Z-score) no capturan, el algoritmo **CUSUM** (Cumulative Sum Control Chart) acumula las desviaciones de la media y alerta cuando la suma acumulada supera un umbral. CUSUM es especialmente efectivo para concept drift gradual: si la calidad de un modelo degrada 0.5% por semana durante dos meses, ningún test diario contra la distribución de referencia alcanza el umbral de significancia, pero CUSUM acumula esas pequeñas desviaciones y alerta cuando el deterioro acumulado es estadísticamente significativo.

La estrategia de alertas efectiva para sistemas de IA combina dos capas. Las **alertas de síntoma** (latencia alta, error rate elevado) disparan inmediatamente con umbrales estáticos bien calibrados: estos síntomas tienen impacto directo en usuarios y deben activar respuesta rápida sin esperar confirmación estadística. Las **alertas de causa raíz** (drift de features, degradación de calidad de modelo, tendencia descendente en fairness metrics) disparan con mayor tolerancia temporal usando umbrales adaptativos o algoritmos de detección de tendencias: no requieren acción inmediata pero necesitan investigación y posiblemente reentrenamiento planificado.

Aspectos técnicos de alertas inteligentes

- **Alertas de síntoma:** latencia p99, error rate, throughput; umbrales estáticos son aceptables para estos porque tienen rangos operativos bien definidos e impacto inmediato en usuarios; requieren respuesta en minutos.
- **Alertas de calidad de modelo:** drift score (KS statistic, PSI), LLM judge score, factual accuracy; requieren umbrales adaptativos porque el baseline cambia con cada reentrenamiento del modelo; tolerancia de horas o días.
- **Detección de anomalías con Z-score:** alertar cuando una métrica supera $\text{mean} \pm 3 \cdot \text{std}$ calculados sobre ventana deslizante de 7-30 días; efectivo para métricas con distribución aproximadamente normal y cambios abruptos.
- **CUSUM:** detecta cambios sutiles y sostenidos que no disparan alertas de Z-score; especialmente útil para concept drift gradual; requiere calibración del parámetro de sensibilidad según la tolerancia al riesgo del sistema.
- **Alert fatigue management:** dead man's switch para alertas de data freshness, agregación de alertas correlacionadas (PagerDuty Grouping o Alertmanager), y

silenciamiento automático durante despliegues planificados; reducir alertas innecesarias es tan importante como detectar problemas reales.

Nota del Arquitecto: *Una alerta que se ignora habitualmente es peor que ninguna alerta: la calibración de umbrales para minimizar false positives es tan importante como la sensibilidad para detectar problemas reales. La práctica de "alert review" trimestral —revisar qué alertas se dispararon, cuáles fueron verdaderos positivos, y ajustar los umbrales de las que generaron falsos positivos— es el mecanismo de mejora continua del sistema de alertas.*

Los sistemas de alertas inteligentes son la infraestructura que garantiza que los problemas de calidad y performance de los modelos en producción son detectados oportunamente y resueltos antes de impactar a los usuarios. La siguiente sección examina las herramientas específicas —Evidently AI, WhyLabs, Langfuse— que implementan este sistema de monitoreo y alertas en la práctica.

Módulo 10 – Capítulo 05 – Sección 05

Herramientas: Evidently AI, WhyLabs y Grafana + Prometheus para LLM

El ecosistema de herramientas de monitoreo para modelos en producción se ha diversificado significativamente en los últimos años, reflejando la bifurcación del mercado entre el monitoreo de modelos de ML clásico (donde las métricas de drift de features estructuradas son el foco) y el monitoreo de LLMs en producción (donde el tracing de prompts y respuestas, y las métricas de calidad específicas de texto, son los problemas centrales). La elección de herramientas debe considerar qué tipo de modelos se opera, qué equipo las va a usar, y si se prefiere una solución managed con menor overhead operativo o una solución open source con mayor control.

Evidently AI es la herramienta open source de referencia para monitoreo de calidad de datos y drift de modelos. Su API Python es directa: `Report(metrics=[DataDriftPreset(), ClassificationPreset()]).run(reference_data=ref_df, current_data=prod_df)` genera un reporte completo de drift por columna con visualizaciones interactivas y tests estadísticos configurables, exportable como HTML o JSON. En su modo de monitoreo continuo, Evidently recibe snapshots periódicos (calculados en el pipeline de inferencia con la librería cliente) y los visualiza en dashboards de evolución temporal que muestran cómo el drift acumulado cambia semana a semana. Evidently es la opción ideal para organizaciones que ya tienen stack de Kubernetes y Grafana, y quieren añadir monitoreo de modelos sin introducir un servicio SaaS adicional, ya que puede deployarse completamente en infraestructura propia.

WhyLabs adopta un enfoque diferente orientado a la privacidad: en lugar de procesar o almacenar los datos de producción en su backend, recibe solo **perfiles estadísticos** calculados localmente por su librería `whylogs`. El comando `logger.log(df)` calcula automáticamente histogramas, quantiles, frecuencias de categorías y métricas de completitud para cada columna, y envía solo estas estadísticas agregadas al backend de WhyLabs, donde se comparan con los perfiles de referencia. Este diseño significa que los datos de producción —que pueden contener PII o información confidencial— nunca salen del entorno de la organización, lo que elimina un obstáculo regulatorio frecuente para el monitoreo de modelos

en sectores financieros o de salud. WhyLabs añade sobre este modelo un dashboard de monitoreo temporal con alertas configurables y un API de consulta que permite queries ad-hoc sobre el historial de perfiles.

Para **LLMs específicamente**, la herramienta open source más adoptada en 2025 es **Langfuse**: traza cada llamada al LLM con su prompt completo, respuesta, latencia, tokens consumidos, costo estimado y metadatos de la aplicación (usuario, conversación, feature que generó el request). Sobre estas traces, Langfuse permite añadir evaluaciones personalizadas — LLM-as-a-judge scores calculados automáticamente, ratings humanos desde la aplicación, y evaluaciones custom via API— que se asocian a cada trace para construir un historial de calidad en producción. La integración de Langfuse con los frameworks más comunes (LangChain, LlamaIndex, OpenAI SDK, Anthropic SDK) requiere en la mayoría de los casos solo un decorator o un callback, minimizando el cambio de código en las aplicaciones que lo adoptan. Langfuse puede deployarse en la infraestructura propia de la organización (Docker, Kubernetes) o usarse como servicio managed.

Grafana + Prometheus con exporters custom siguen siendo el stack de monitoreo más adoptado para métricas de sistema, y pueden extenderse para métricas de modelos creando métricas custom en la librería `prometheus_client` de Python que exponen las métricas de calidad del modelo (drift score, LLM judge score, error rate por tipo) junto a las métricas de infraestructura (latencia, throughput, GPU utilization) en los mismos dashboards. Este enfoque tiene la ventaja de que consolida todo el monitoreo del sistema en un único stack conocido por el equipo de operaciones, sin introducir nuevas herramientas a gestionar.

Comparación de herramientas de monitoreo

- **Evidently AI**: open source, especializado en drift y calidad de datos/modelos, reportes HTML y JSON exportables, integrable en pipelines de Airflow o Prefect como step de validación automático; deployment self-managed en Kubernetes.
- **WhyLabs**: SaaS, ingesta de perfiles estadísticos (no datos crudos), privacidad por diseño, UI de monitoreo temporal con alertas nativas; para organizaciones con requisitos de privacidad de datos estrictos.
- **Grafana + Prometheus**: stack generalista extendida para IA mediante exporters custom; consolida el monitoreo de infraestructura y de modelos en el mismo stack; requiere más desarrollo pero elimina herramientas adicionales.
- **Langfuse**: open source (self-hostable), para observabilidad de LLMs; tracing de prompts y respuestas, evaluaciones, costos por modelo; integración nativa con LangChain,

LlamaIndex y OpenAI/Anthropic SDKs; la opción de referencia para aplicaciones de LLM.

- **Arize AI (Phoenix):** plataforma de observabilidad para LLMs y ML clásico; embeddings visualization para detectar clustering drift, tracing de llamadas LLM, evaluaciones de hallucination y relevance; especialmente útil para debugging de sistemas RAG.

Nota del Arquitecto: Ninguna herramienta de monitoreo resuelve el problema de calidad de datos: son instrumentos de detección, no de prevención. El monitoreo efectivo requiere también un proceso definido de respuesta cuando las alertas se disparan. La cadena completa es: métricas → alertas → runbook de investigación → decisión (ignorar / investigar / reentrenar / rollback) → acción. Sin el proceso de respuesta, el monitoreo produce datos que nadie usa.

Las herramientas de monitoreo de modelos en producción son componentes de la plataforma que deben ser operados con el mismo rigor que los endpoints de serving. La sección de cierre de este capítulo sintetiza la distinción fundamental entre monitorear una aplicación convencional y monitorear un sistema de IA, y establece el principio que debe guiar el diseño del sistema de monitoreo desde antes del primer despliegue.

Módulo 10 – Capítulo 05 – Sección 06

Cierre: monitorear un modelo en producción es diferente a monitorear una aplicación tradicional

Una aplicación web tradicional tiene un estado binario bien definido desde el punto de vista de la calidad: o retorna un HTTP 200 con el contenido esperado, o retorna un error. Un modelo de IA en producción puede retornar un HTTP 200 con latencia correcta y sin errores de infraestructura mientras produce predicciones degradadas que nadie detecta hasta que el impacto en el negocio es visible: la tasa de conversión cae semanas después, los usuarios empiezan a reportar respuestas incorrectas, o un auditor identifica que el modelo está produciendo resultados sistemáticamente sesgados para ciertos subgrupos. Esta es la diferencia fundamental que determina todo el diseño del sistema de monitoreo.

El monitoreo de modelos en producción requiere dos capas complementarias que deben coexistir. La **capa de infraestructura** —latencia, throughput, error rate, GPU utilization— puede monitorizarse con las mismas herramientas y los mismos procesos que cualquier microservicio. Grafana y Prometheus son suficientes para esta capa; el equipo de operaciones ya los conoce; las alertas tienen significado claro y acción obvia. La **capa de calidad del modelo** —drift de datos, métricas de calidad de predicciones, fairness metrics, coherencia y fidelidad de LLMs— requiere instrumentación específica, y en muchos casos requiere acceso a ground truth o a evaluadores que produzcan los labels correctos en producción. Esta segunda capa es la que la mayoría de las organizaciones tienen incompleta o ausente, y es la que produce los "silent failures" más costosos.

El concepto de **fallo silencioso** es el más importante de este capítulo. Un modelo puede seguir respondiendo correctamente desde el punto de vista de la infraestructura —HTTP 200, latencia dentro del SLO, sin errores de sistema— mientras su calidad se degrada gradualmente por concept drift, cambios en los datos de entrada, o comportamientos emergentes no anticipados. Solo el monitoreo activo de métricas de calidad puede detectar estos fallos. Sin ese monitoreo, el fallo silencioso se convierte en un fallo visible solo cuando el impacto en el negocio es significativo: una regresión de semanas de duración que habría tomado días de investigar si se hubiera detectado temprano ahora requiere semanas de

investigación forense y potencialmente afecta a decisiones de negocio que se tomaron sobre predicciones incorrectas.

El monitoreo efectivo de un modelo en producción comienza a diseñarse antes del despliegue, no después de un incidente. Las preguntas que determinan el diseño del sistema de monitoreo son: ¿qué métricas de calidad se recolectan y cómo? (¿el sistema produce outputs evaluables automáticamente o necesita evaluadores humanos?), ¿cómo se obtiene el ground truth? (¿es inmediato, como un click, o llega con retraso, como una conversión 30 días después?), ¿cuál es el proceso de respuesta cuando la calidad cae? (¿reentrenamiento automático, rollback manual, escalado al equipo de negocio?). Sin respuestas a estas preguntas antes del despliegue, el monitoreo de calidad nunca se implementa porque siempre hay algo más urgente que atender después del lanzamiento.

Principio rector

El monitoreo de un modelo en producción debe comenzar a diseñarse antes del despliegue: qué métricas de calidad se recolectan, cómo se obtiene el ground truth, y cuál es el proceso de respuesta cuando la calidad cae son preguntas que deben responderse en el diseño del sistema, no después de un incidente. El monitoreo que se añade como afterthought siempre cubre solo la capa de infraestructura, la más fácil de implementar y la menos específica de los problemas reales de los sistemas de IA.

"The goal is to turn data into information, and information into insight."

— Carly Fiorina, ex CEO de Hewlett-Packard, cuya distinción entre datos, información e insight aplica directamente al sistema de monitoreo: las métricas son datos; las alertas bien calibradas son información; la decisión de reentrenar o hacer rollback es insight operacional.